Running a simulation on SDSC Expanse with simrun

pmsimstats team

Running a simulation on SDSC Expanse with simrun

2026-11-03 15:59 PST

1. What this vignette covers

This vignette walks a new lab member through one complete simulation job on SDSC Expanse: getting an account and logging in, sending the R code and its software environment to the cluster, submitting the job, watching it, and bringing the results back to the laptop. The worked example is the G10 planning run of paper 01, driver 19 (`19-g9-planning-and-fig4b.R` in `analysis/scripts/quick-sim/carryover-closed-form/`), but every step applies to any script in a zzcollab compendium.

The lab sends jobs with `simrun`, a small command-line tool that also drives the lab's own servers (zeus and mercury). On Expanse it does four things for you:

1. **Code.** It pushes the commit you name to a git repository in your Expanse storage. Jobs always run a commit, never your working folder, so a result can always be traced to the code that made it.
2. **Software environment.** It sends the project's Docker image (`--ship`), which Expanse converts once to a Singularity image (`.sif`) and reuses for later jobs with the same `Dockerfile` and `renv.lock`.
3. **The job.** It submits a SLURM batch job on the `shared` partition, charged to the lab's allocation.
4. **Results.** It copies back every file the job created or changed, with the job log and a provenance record (`simrun fetch`), and on request lays them into the project where a local run would have put them (`simrun apply`).

You never copy files to Expanse by hand and never write a SLURM script.

Status of this route (read first). Every piece below was checked on its own on 2026-10-26: login, the SLURM account and partition, the Singularity module, core allocation and a speed benchmark. As of this writing no project job has yet run end to end through simrun on Expanse. Treat your first job as a test: submit something small (Section 7) before a production run, and report anything that differs from this vignette.

2. Should the job go to Expanse at all?

The lab's servers cost nothing to use and need no second login factor. Use Expanse when they are busy or when a job needs many more cores.

	zeus (lab)	Expanse
Cost	none	core-hours from a hard-capped allocation
Login	SSH key	password or key, plus a TOTP code
Cores per job	about 8 is best	up to 128 on the `shared` partition
Speed per core	close to an M1 laptop's for one worker; about a third of that per worker at 8 workers	about half of an M1's (base-R benchmark)
Jobs at once	one at a time	many, subject to the queue
Image	built on the server or shipped	must be shipped (`--ship`)

The allocation is project **SDS154**, a trial allocation of 1,000 core-hours from 2026-10-20 to about 2027-04-20. It is a hard cap: when it is spent, jobs stop being accepted. A careless job can use a large share of it, so size jobs from a timing run (Section 6.2).

3. One-time setup: accounts and login

3.1 An ACCESS account on the allocation

Expanse is allocated through ACCESS (`access-ci.org`).

1. Create an ACCESS account at `access-ci.org` if you do not have one. Your ACCESS username becomes your Expanse username; this vignette writes it as `<access-id>`.

2. Ask the allocation's PI to add you to project SDS154 in the ACCESS allocations portal. Whether a trial allocation accepts additional users has not been checked; if it does not, the PI will need to request a regular allocation.
3. Wait until the account is active on Expanse; the login in Section 3.3 is the test.

3.2 Two-factor login (TOTP)

Expanse requires a time-based one-time code at every new SSH connection. Enroll once at `passive.sdsc.edu` with an authenticator app (Google Authenticator, 1Password, or similar). The lab's Expanse login uses this SDSC TOTP, not Duo.

3.3 An SSH key and an `expanse` alias

A key replaces the password step; the TOTP code is still asked for. On the Mac:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_expanse \
  -C expanse-<access-id>
```

Add this entry to `~/.ssh/config`:

```
Host expanse
    HostName login.expanse.sdsc.edu
    User <access-id>
    IdentityFile ~/.ssh/id_ed25519_expanse
    IdentitiesOnly yes
    ControlMaster auto
    ControlPath ~/.ssh/control-%r@%h:%p
    ControlPersist 4h
```

The last three lines matter. `simrun` runs many short SSH commands, and without them each would ask for a TOTP code. With them, one login by hand opens a shared connection that every later command reuses for up to four hours.

Log in once with your password and code, and install the key:

```
ssh expanse
mkdir -p ~/.ssh && chmod 700 ~/.ssh
cat >> ~/.ssh/authorized_keys     # paste the contents of
                                  # ~/.ssh/id_ed25519_expanse.pub,
```

```
                                    # then press Ctrl-D
chmod 600 ~/.ssh/authorized_keys
exit
```

From now on `ssh expanse` asks only for the TOTP code.

3.4 The simrun server programs on Expanse

simrun's Expanse side is two shell scripts kept in the lab's dotfiles repository (`share/simrun/simrun-server-slurm` and `share/simrun/simjob-slurm`; ask the PI for access). Copy them into `~/.local/bin` on Expanse under the names simrun expects:

```
ssh expanse true                    # answer the TOTP prompt once
ssh expanse mkdir -p .local/bin
scp ~/dotfiles/share/simrun/simrun-server-slurm \
  expanse:.local/bin/simrun-server
scp ~/dotfiles/share/simrun/simjob-slurm expanse:.local/bin/simjob
ssh expanse chmod +x .local/bin/simrun-server .local/bin/simjob
```

The scripts charge jobs to account `sds154` on partition `shared` with QOS `shared-normal`, and always add `--nodes=1` (Expanse rejects a job without it) and `--constraint=lustre` (only those nodes can see the storage below). A user on a different allocation sets `SIMRUN_SLURM_ACCOUNT` in the Expanse `~/.bashrc`.

3.5 Where jobs live on Expanse

Jobs, images and data go on Lustre **project** storage. The 100 GB home quota is too small for images, and Lustre **scratch** is purged after 90 days, which is shorter than the allocation. Log in, find the project directory, and set two variables in the Expanse `~/.bashrc`:

```
ssh expanse
ls /expanse/lustre/projects/sds154/    # confirm the path
mkdir -p /expanse/lustre/projects/sds154/<access-id>/simrun/data
cat >> ~/.bashrc <<'EOF'
export SIMRUN_ROOT=/expanse/lustre/projects/sds154/<access-id>/simrun
export SIMRUN_DATA_ROOT=$SIMRUN_ROOT/data
EOF
exit
```

The exact subpath under `projects/sds154/` was still to be confirmed when this was written; use whatever `ls` shows. Check that the variables reach a non-interactive shell, which is how simrun calls the server:

```
ssh expanse 'echo $SIMRUN_ROOT $SIMRUN_DATA_ROOT'
```

Both paths must print. If they are empty, put the `export` lines above any early `return` in `~/.bashrc` (some default `.bashrc` files stop early for non-interactive shells).

Containers run with Singularity. On Expanse the module is `singularitypro` (not `apptainer` or `singularity`); the server script loads it itself.

3.6 The Mac side

You need:

- **Docker Desktop**, running, to build the project image.
- **`simrun`** on your `PATH` (`dotfiles/bin/simrun`, linked into `~/bin`).
- **`zstd`** (`brew install zstd`), optional: it roughly halves the image upload.
- **A simrun config file**, `~/.config/simrun/config` (start from `dotfiles/share/simrun/config.example`), with modest Expanse defaults so that a forgotten option cannot spend the allocation:

```
cores@expanse = 4
mem@expanse = 16
time@expanse = 02:00:00
```

3.7 Check the connection

```
ssh expanse true                    # TOTP prompt
simrun --host expanse status
```

An empty list means everything is connected. `simrun --host expanse config` shows the settings in effect and where each came from.

4. Preparing a project (once per project, then per change)

4.1 The script

The script needs no simrun- or SLURM-specific code, with two habits:

- **Size parallel workers from `SIM_CORES`.** The job sets it to the cores SLURM gave the job. `parallel::detectCores()` inside the container reports the whole node (128 cores), and starting that many workers on a 16-core allocation oversubscribes the CPUs and slows the job down (measured on Expanse).

```
cores <- as.integer(Sys.getenv('SIM_CORES', '1'))
res <- parallel::mclapply(seq_len(n_reps), one_rep, mc.cores = cores)
```

Driver 19 reads `SIM_CORES` and also takes `--cores`; pass the same number to both simrun and the script.

- **Write outputs inside the project** (under `analysis/`), and create the output folder in the script: git does not store empty folders, so the job's fresh checkout may not have it. Driver 19 writes to `analysis/data/quick-sim/carryover-closed-form/published-planning/`.

Run the script locally at a tiny size first (driver 19: `--smoke`).

4.2 Commit

simrun refuses to submit while tracked files have uncommitted changes, so that nobody mistakes a result for one that includes them.

```
git status
git add <files>
git commit -m '...'
```

`--commit HEAD` overrides the check and runs the last commit without your local edits; use it only on purpose.

4.3 Build the image

Expanse cannot build images, so you build it on the Mac and simrun ships it. Expanse nodes are x86-64, and a Mac with Apple silicon builds ARM images by default. The project Makefile already sets `PLATFORM = linux/amd64`:

```
make docker-build
```

The image is labeled with a hash of `Dockerfile` and `renv.lock`. Before shipping, simrun checks that the label matches the commit being submitted and refuses if it does not, so rebuild after any change to either file. The pmsimstats-ng image is about 5.5 GB; the first shipment of a new environment takes a while, and later jobs with the same environment reuse the `.sif` on Expanse.

4.4 Data not in git

Git carries the code. Files the job needs that are not in git (raw data, large inputs) go to Expanse once, and again whenever they change:

```
simrun --host expanse data push analysis/data
```

Every job starts from the commit with these files laid over it. Driver 19 needs no untracked data: its inputs are committed.

5. Each working session

```
ssh expanse true                    # TOTP; opens the 4-hour connection
ssh expanse expanse-client user -r expanse    # allocation balance
```

If a `simrun --host expanse` command hangs at a prompt or fails to connect later in the day, the four hours have passed: run `ssh expanse true` again.

6. Submitting

6.1 The command

```
D19=analysis/scripts/quick-sim/carryover-closed-form
D19=$D19/19-g9-planning-and-fig4b.R
simrun --host expanse submit --ship \
  --cores 16 --mem 32 --time 10:00:00 --label p01-g10-pf -- \
  Rscript $D19 \
    --params published --specs E7null_cr2,E7cr2 --block all --cores 16
```

Everything before `--` is for simrun; everything after it is the command the job runs, from the project root, inside the container. `D19` only shortens the line: your shell expands it before simrun sends the command. The commands below reuse it.

Option	Meaning
`--host expanse`	Send to Expanse (the SSH alias of Section 3.3). Give it to every command about this job.
`--ship`	Send the local image. Required on Expanse.
`--cores N`	CPUs reserved from SLURM (`--cpus-per-task`); sets `SIM_CORES`.

Option	Meaning
`--mem G`	Memory in GB for the whole job.
`--time HH:MM:SS`	Wall-clock limit. The job is killed when it is reached.
`--label TEXT`	A name that appears in `status`.

simrun prints the job number:

```
job 3 queued on expanse  (.../pmsimstats-ng @ 4e1c2a9, 16 cores, 32 GB)
  simrun watch 3    simrun log 3    simrun fetch 3
```

Job numbers are per host: job 3 on Expanse is unrelated to job 3 on zeus.

6.2 Sizing a job

- **Cores.** Choose from a timing run, not by default. Throughput per core falls as cores rise (on Expanse, 33 times one core at 60 cores and 57 times at 120, in the benchmark), and a job with a long tail of slow cells leaves cores idle at the end. Sixteen is a reasonable start for driver 19.
- **Memory.** Request about 2 GB per core. On the `shared` partition a job is believed to be charged by the larger of its cores and its memory as a share of a node (about 2 GB per core), so 32 GB for 16 cores costs the same as 16 cores, while 64 GB would be charged as 32 cores. This charging rule has not been confirmed in the Expanse user guide; check it before large jobs.
- **Time.** Set about twice the projected wall time. SLURM charges the time used, not the limit, but a job that reaches the limit is killed and its work is lost: a resubmitted job starts from a fresh checkout in a new folder, so a driver's checkpoints from the killed job are not reused. The `shared` partition allows up to 48 hours.
- **Projection.** Base it on a stored run of the same work where one exists. Driver 19's block P took 5.4 hours per analysis on zeus at 8 cores (jobs 40 and 53), about 2.2 core-seconds per replicate. Its local timing mode (`--time`, one worker) predicted about a quarter of that: a single worker is fast, but zeus keeps only about a third of that speed when 8 workers run together (measured 2026-11-03, `docs/74` Section 2). A one-worker timing run therefore understates a multi-worker job severalfold, by an amount that depends on the host. How Expanse scales with 16 workers on this code is unknown; the first Expanse job should record its core-hours per replicate, and until then a projection should start from the zeus figure of about 134 zeus core-hours for the two driver 19 jobs.

Core-hours charged are roughly cores times elapsed hours: a 16-core job that runs 3 hours costs about 48 of the 1,000.

6.3 Several jobs at once

Expanse runs jobs in parallel, so independent jobs can be submitted together. Driver 19's block N job writes to its own folder (`published-planning/block-N/`), so it can run beside the first:

```
simrun --host expanse submit --ship \
  --cores 16 --mem 32 --time 03:00:00 --label p01-g10-n80 -- \
  Rscript $D19 \
    --params published --specs E7null_cr2,E7cr2 --block N --cores 16
```

Two jobs must not write to the same output folder.

Results from Expanse are not draw-for-draw identical to results from zeus or the Mac (different numerical libraries). Analyses that must be compared trial by trial have to run on the same host.

7. Watching a job

```
simrun --host expanse status          # all jobs
simrun --host expanse watch 3         # refreshes until the job ends
simrun --host expanse log 3 -f        # the job's output as it runs
```

State	Meaning
`queued`	Waiting in the SLURM queue (`PENDING`)
`running`	Running in its container
`done`	Finished with exit status 0
`failed`	Finished with any other exit status
`cancelled`	Removed before it started

Expanse jobs never show `building`: the image is already there. Waiting in `queued` for minutes to hours is normal on a shared cluster. You can close the laptop while a job runs; you need the connection again only to check on it or fetch it.

A first test job. Before a production run, submit a two-minute job to check the whole route:

```
simrun --host expanse submit --ship --cores 2 --mem 4 --time 00:15:00 \
  --label smoke -- \
  Rscript $D19 \
    --params published --specs E7null_cr2 --block F --smoke --cores 2 \
    --out-dir analysis/data/expanse-smoke
```

It costs well under one core-hour, ships the image (the long part), and leaves the `.sif` in place for the real jobs.

8. Bringing results back

8.1 Fetch

```
simrun --host expanse fetch 3
```

This copies the job's results into `analysis/remote_runs/job-3-<commit>-expanse/` on the Mac and changes nothing else. The folder holds:

- `results/`: every file the job created or changed, at its path in the project;
- `job.log`: everything the script printed;
- `provenance.yml`: commit, command, image digest, cores, memory, start and finish times, exit status.

`analysis/remote_runs/` is in `.gitignore`; fetched runs are records, not part of the repository.

8.2 Apply

When the results look right, lay them into the project where a local run would have put them:

```
simrun --host expanse apply 3
```

`apply` refuses, and copies nothing, if any result would overwrite a local file that differs from it. Compare the two first; `--force` overwrites.

8.3 Check before using

For driver 19, before rendering paper 01:

- `meta.rds` records the parameter set (`params$param_set` is `published`) and both analyses (`spec`).
- Each output folder's `PARAM_SET` file says `published`; paper 01 reads nothing from a folder without it.
- The log reports all (cell, replicate) jobs present.
- Then render (below). The bold [pending] markers disappear when the files are in place.

```
bash tools/render.sh \
  analysis/report/01-dgp-mean-moderation-vs-mvn/report.Rmd
```

8.4 Tidy up on Expanse

Job folders stay on Lustre until removed, and count against the project's storage:

```
simrun --host expanse clean 3          # one job, after fetching it
```

9. When something goes wrong

Symptom	Cause and fix
A prompt for a verification code on every command	The shared connection is missing or expired: `ssh expanse true`, and check the `Control*` lines of Section 3.3
`uncommitted changes ...`	Commit first (Section 4.2)
`no local image ...`	Run `make docker-build`
`built from a different Dockerfile/renv.lock`	The image is older than the commit: rebuild it
Exit 20 at once	No `.sif` on Expanse and `--ship` was missing; resubmit with `--ship`
`set SIMRUN_ROOT in ~/.bashrc`	The variables of Section 3.5 do not reach a non-interactive shell
`bank_limit plugin: Please set the nodes parameter`	An old server script without `--nodes=1`; reinstall it (Section 3.4)
`execve(): singularity: No such file or directory`	The `singularitypro` module was not loaded; the current server script loads it
Job slower than expected	More workers than `--cores` (oversubscription): use `SIM_CORES`
Exit 137	Out of memory: raise `--mem`, or lower the cores
Job killed at the time limit	Submit again with a larger `--time` (`simrun --host expanse resubmit 3` reruns the same commit and command with the same limit, so it would be killed again); the new job starts from scratch
Stuck in `queued`	Normal under load; `ssh expanse squeue -u <access-id>` shows SLURM's view and reason
An R error (exit 1)	`simrun --host expanse log 3`; fix, commit, rebuild if needed, resubmit

To stop a job: `simrun --host expanse cancel 3`.

10. Command summary

Step	Command
Open the session	`ssh expanse true`
Balance	`ssh expanse expanse-client user -r expanse`
Build the image	`make docker-build`
Push untracked data	`simrun --host expanse data push analysis/data`
Submit	`simrun --host expanse submit --ship --cores N --mem G --time HH:MM:SS --label L -- CMD`
Status	`simrun --host expanse status`
Follow	`simrun --host expanse watch ID`, `simrun --host expanse log ID -f`
Fetch	`simrun --host expanse fetch ID`
Apply	`simrun --host expanse apply ID`
Rerun	`simrun --host expanse resubmit ID`
Stop	`simrun --host expanse cancel ID`
Remove from Expanse	`simrun --host expanse clean ID`

11. How each statement here was established

- **Verified on Expanse (2026-10-26):** login with a password or key plus TOTP; the account string `sds154`, partition `shared` and QOS `shared-normal`; the need for `--nodes=1` and for loading `singularitypro`; the module name; core reservation through `--cpus-per-task` and the oversubscription slowdown; the per-core speed relative to an Apple M1 laptop (julia).
- **Inspected (read in the simrun source):** the commands, options, folder names, the `--ship` hash check, the SLURM options the server script passes, and what `fetch` and `apply` do.
- **Not yet verified:** a complete simrun job on Expanse; the exact Lustre project path; adding a second user to the trial allocation; the memory-share charging rule; Expanse's speed on the project's simulation code (only a base-R benchmark was run there), and so the core-hour estimate of Section 6.2. Whoever runs the first job should update this section.

General simrun usage on the lab servers (zeus and mercury) is documented in the lab's simrun guide:

- `dotfiles/share/simrun/vignette.md`